2021 IEEE Intelligent Transportation Systems Conference (ITSC)
Indianapolis, USA. September 19-21, 2021

An Empirical Study on Ego Vehicle Trajectory Prediction for Bicycles in Urban Environments Based on Conditional Imitation Learning

Andreas Weißmann, Daniel Görges

2021 IEEE International Intelligent Transportation Systems Conference (ITSC) | 978-1-7281-9142-3/21/$31.00 ©2021 IEEE | DOI: 10.1109/ITSC48978.2021.9564999

***Abstract*— This paper presents the results of an empirical study on a vision-based trajectory prediction approach for usage in urban environments based on conditional imitation learning with focus on bicycles. The approach uses neural networks to predict the changes in heading and speed of an ego vehicle for a specified prediction horizon and sample time which can be shared over V2X for collision avoidance or used in further advanced driver assistance systems. Predicting changes allows a location-independent trajectory representation based on the vehicle's current speed and heading. The prediction is carried out for the four predefined maneuvers "turn left", "turn right", "go straight" and "lane follow". Several neural networks with different input data configurations are evaluated for the same prediction task. The available network inputs are images from a front-facing camera, heading information, vehicle speed and turning indicator signals. For training the networks a dataset was created from several hours of riding an electric bicycle in structured and unstructured environments of a city. In addition the networks have been implemented on an embedded GPU platform to investigate the real-time capability of the presented approach. The networks were configured and trained to predict five seconds into the future with a sample time of $0.2\,\mathrm{s}$. The results of each network are presented and compared against each other. The results show that the implemented networks are capable of predicting a trajectory in short to mid-term ($1-3\,\mathrm{s}$) with good accuracy while having difficulties predicting turn maneuvers in long-term range ($3-5\,\mathrm{s}$) due to the lack of road layout information.**

I. Introduction

Predicting the future trajectory of vehicles is a very important task of many functionalities used in advanced driver assistance systems (ADAS) and automated vehicles. The knowledge can be used in different ways, e.g. for collision avoidance or comfort decisions. Trajectory prediction mostly focuses on motorized traffic. However, due to the shared environment, reacting to cyclists and including them in assistance and safety functions has become an increasing interest over the past years. With the success of the electric bicycle even more cyclists will be present on the road which motivates further research to provide a safe traffic environment. While our presented method is intended and evaluated for an electric bicycle, the method itself is in fact vehicle-independent. Much research has been devoted to trajectory prediction for different environments and situations in which a car detects and tracks the surrounding traffic participants using sensors such as radar, lidar and camera. Based on this information the future trajectories of the traffic participants are predicted and can then be used for e.g. lane change assistance, collision avoidance, etc. The prediction task is often studied for motor vehicles in well structured environments such as highways [1], [2], [3], [4]. However, with the progress in automated driving the focus started to shift towards handling scenarios with pedestrians and bicycles in urban environments. Over the last years various methods for trajectory prediction in urban environments have been presented. As many accidents take place at intersections and junctions (e.g 32.4% in Germany [5]), many approaches focusing at these scenarios have been introduced. For instance, equipping intersections with stationary sensor setups and creating prediction models from the observed traffic participants is an often studied approach [6], [7], [8], [9]. However, methods and approaches to tackle the trajectory prediction task for different traffic participants and environments can be very versatile depending on the scenario. Recent surveys that address this variety are given in e.g. [10], [11], [12].

Generally the approaches can be classified into (i) physics-based methods where dynamic vehicle models are used for trajectory prediction, (ii) pattern-based methods where dynamic function approximators trained from observed vehicle trajectories are used for trajectory prediction, and (iii) planning-based methods where the driving policies and motion intents are used for trajectory prediction [12]. The approaches differ in terms of prediction accuracy, data requirements, computational complexity, etc. For a detailed discussion the readers are refered to the surveys indicated above.

The approach presented in this paper extends the idea of learning a driving policy from observations and therefore combines planning-based and pattern-based methods. In recent years imitation learning has become a very popular approach to learn driving policies due to its ease of use. In imitation learning a neural network is trained on many hours of expert driving, resulting in a driving policy similar to that one observed during training. The methods use sensor inputs to observe the surrounding vehicles and input an image or detections along with different vehicle states such as steering angle, vehicle speed, acceleration, etc. to a neural

This work was supported by the German Federal Ministry of Transport and Digital Infrastructure from resources of the 2020 National Cycling Plan under grants VB1603 and VB1905.

Andreas Weißmann is with the Electromobility Research Group, Department of Electrical and Computer Engineering, University of Kaiserslautern, 67663 Kaiserslautern, Germany (e-mail: weissmann@eit.uni-kl.de).

Daniel Görges is with the Geman Research Center for Artificial Intelligence (DFKI), 67663 Kaiserslautern, Germany (e-mail: daniel.goerges@dfki.de).

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

network which calculates an action (e.g. control input for steering, brake or acceleration) for the vehicle at the current time instant. One early example is [13] in which a neural network was trained to control the steering of a vehicle for lane following where only images of cameras along with the current steering angle were used to predict the new desired steering angle for vehicle control. Another recent approach that introduced conditional imitation learning was presented in [14], where "conditional" refers to using commands to "switch" network behavior. One proposed implementation was using sub-networks commonly referred to as branches for specific commands, e.g. "turn left", "turn right", etc. These sub-networks get the same input data but are trained only on the groundtruth of the respective maneuver. It was shown that this leads to better results than using only one branch for all commands. The approach uses an image along with the current speed as input to predict the desired steering angle and acceleration. The method was evaluated in a simulated environment as well as with a model car in a residential area. The authors of [15] also use conditional imitation learning to predict two waypoints from which the steering control of the vehicle is derived. An evaluation of the method was given for a simulation environment and in addition for model cars in different environments. Another recent approach using imitation learning was presented in [16] which also predicts commands to control a vehicle but uses four images as input, one pair of stereoscopic images for the perception of depth and one pair of past images for the perception of motion. The approach not only predicts the next control but also predicts the vehicle control commands for a short time span of $330\,\mathrm{ms}$ divided into ten time steps each $33\,\mathrm{ms}$ apart from each other forming a planned trajectory. The evaluation of this method was done with model cars. Similar to the aforementioned approach due to the different goal no results or considerations regarding trajectory planning was given.

Many successful experiments in the aforementioned literature have shown that neural networks are capable of learning the complex tasks necessary to control a vehicle in different environments. This motivates further research by reformulating the goal of the network from learning a driving policy to predicting the future trajectory of an ego vehicle, which is the main focus of this work.

In this paper a method for trajectory prediction is proposed which can be run in real-time on resource-constraint computing platforms using only basic sensors. The method provides a prediction of the trajectory of the ego vehicle which can be used for ADAS in the ego vehicle or shared with other vehicles and the infrastructure using V2X e.g. for cooperative collision avoidance. The contribution of the paper not only consists in introducing the method but also in analyzing its properties in an empirical study. Particularly the causes for prediction errors are investigated and possible solutions are elaborated. Specifically the influence of different environments is studied. This is important for vehicles which are used in versatile environments such as bicycles. The findings not only apply to the method presented in the paper but also to other pattern-based methods and are thus relevant in a wider scope. In the paper several neural network architectures used for long-term trajectory prediction for a bicycle via conditional imitation learning are presented using different combinations of input data. The influence of these different input combinations as well as the influence of the different network architectures on the prediction accuracy is investigated. A dataset gathered from riding a bicycle in a city is used to train these networks with realistic scenarios. To investigate the real-time capability of the networks the execution time on an NVIDIA Jetson TX2 embedded GPU platform is evaluated.

The paper is organized as follows. In Section II the details of the approach are presented. The section covers a scenario overview, the representation of the predicted trajectories as well as the network architecture details along with the dataset and training details. The results are given in Section III.

II. Methodology

A. Scenario Overview

The proposed method is intended to be implemented on a bicycle and predict its long-term trajectory in terms of heading changes $\Delta\varphi$ and speed changes Δv for a given time horizon with a sample time of Δt. The trajectory can then be calculated recursively over the predicted time horizon for every Δt with the predicted $\Delta\varphi, \Delta v$ starting from the current vehicle yaw, speed and position. These trajectories can be used in onboard ADAS in the bicycle or sent via V2X interfaces to surrounding vehicles or infrastructure to aid in collision avoidance tasks. The trajectory prediction is implemented as a neural network trained via conditional imitation learning from several hours of riding a bicycle equipped with various sensors and a turn signal in an urban environment. The used data are images from a front-facing camera as well as measurements of the vehicle speed and vehicle yaw. The neural network uses four output branches trained for the following maneuvers: "lane follow" as default branch, "turn right" and "turn left" for the respective actions at intersections or driveways and "straight" for going straight at an intersection. These maneuvers are considered to be known beforehand to choose the correct branch. During

TABLE I
Overview of models and the respective inputs

	Input		
Model name	Image	Current speed	Past measurements
FC V1			**X**
FC V2		**X**	**X**
Visual small V1	**X**	**X**	
Visual small V2	**X**		**X**
Visual small V3	**X**	**X**	**X**
Visual deep V1	**X**	**X**	
Visual deep V2	**X**		**X**
Visual deep V3	**X**	**X**	**X**

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

creation of the dataset these maneuver inputs were generated by using the turn signals of the bicycle. To analyze the influence of the different available sensor data on the resulting trajectory prediction, several models were trained with different input combinations. To validate the findings regarding the benefits of using images as additional input, the experiments were carried out for different architectures. One model type corresponds to a simple fully connected (FC) network only using the sensor data. The other model types are one "small" and one "deep" neural network both using convolutional layers for images and fully connected layers for sensor data. All of the combinations are shown in Table I. The network details (layers, activation functions, ...) are given in the respective section.

B. Representation of Predicted Trajectories

To avoid learning location-specific features from the training data by using absolute values, trajectories are represented in terms of speed and vehicle yaw changes. Using the yaw instead of the steering angle leads to an abstraction of the actual vehicle and a more generic representation and thus avoids additional complexity in the learning task for different vehicles. This can be seen when considering the movement of bicycles which is not only dependent on the steering angle but also on the lean angle in contrast to cars or trucks. Therefore using the yaw instead combines these two, resulting in a simplified learning task.

The predicted trajectory is created recursively from predicting the changes in yaw and speed $\Delta\varphi_k, \Delta v_k$ at discrete-time grid points P_k over the entire prediction horizon, assuming $\Delta\varphi_k, \Delta v_k$ to be constant over the sample time Δt. An illustration of this is shown in Fig. 1 where $\Delta\varphi_0, \Delta v_0$ is the predicted yaw and speed change between the grid points P_0 and P_1. Note that the absolute value for yaw and speed at the initial point P_0 is known from measurements. Therefore the trajectory can be created recursively from this known point. For the implementation, a prediction horizon of $5\,\mathrm{s}$ was chosen with a sample time $\Delta t = 0.2\,\mathrm{s}$ leading to 25 grid points for the trajectory. It is to be mentioned that $0.2\,\mathrm{s}$ is suitable for a bicycle application due to its low speed profile. However, a smaller Δt is preferable and may be a necessity for higher speeds.

To create the trajectory in an x, y representation we use

$$\begin{aligned} x_{k+1} &= x_k + v_k \cdot \cos(\varphi_k) \cdot \Delta t \\ y_{k+1} &= x_k + v_k \cdot \sin(\varphi_k) \cdot \Delta t \end{aligned} \tag{1}$$

and

$$\begin{aligned} \varphi_{k+1} &= \varphi_k + \Delta\varphi_k, \quad \varphi_k = \varphi_0 \\ v_{k+1} &= v_k + \Delta v_k, \quad v_k = v_0 \end{aligned} \tag{2}$$

with φ_0 and v_0 being the initial heading and speed respectively.

C. Dataset

The dataset used for creating the training and test data consists of 5 hours of riding a test bicycle equipped with a front-facing camera, IMU, steering sensor and a turning signal in various structured and unstructured environments

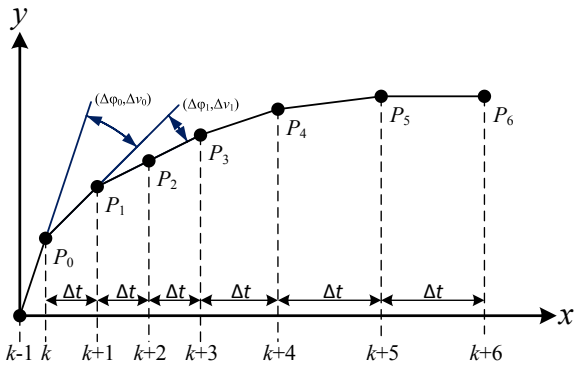


Fig. 1. Trajectory representation

in the city of Kaiserslautern. Due to resource constraints of the hardware, the sample time for logging the sensor measurements of the system was $0.2\,\mathrm{s}$. The logged data were converted to samples for each maneuver $m \in \{\text{lane follow}, \text{turn right}, \text{turn left}, \text{straight}\}$ containing the inputs: image I and sensor data S_d as well as the groundtruth for predicting the trajectories in form of $(\Delta\varphi_k, \Delta v_k)$ pairs over the specified prediction horizon as described in the previous section. For conversion to training data the values of $\Delta\varphi_k$ and Δv_k are normalized to a range of ± 1 according to the highest observed speed and yaw change, with an additional 10% reserve. The "turn left/right" and "straight" maneuvers consist of approaching an intersection or driveway, the turning itself and leaving the intersection. The "lane follow" maneuver contains all samples not related to turning or intersections. Therefore it acts like a "catch all" maneuver. The gathered data contains a variety of different environments and interactions with other traffic participants, pedestrians and obstacles. Some examples of the dataset are shown in Fig. 2. These are some examples for the different infrastructures found in the dataset. The examples correspond to the maneuver lane follow. Additional examples related to turning maneuvers will be shown and discussed in the results Section III.

D. Network Details

Depending on the corresponding model presented in Table I the network architecture differs a little. The "visual small" and "visual deep" models consist of a visual path and a sensor path that is concatenated and then evaluated from the respective branch of the active maneuver. A representation for these models is given in Fig. 3.

The Res Block differs for the visual small and the visual deep architecture. The visual small model versions use simple residual building blocks introduced in [17]. The visual deep model versions use the building block introduced in [18] (with cardinality=32) and in addition two of these blocks are used consecutively to increase the network depth further. The structure of these blocks is depicted in Fig. 4. The decision to exchange some conventional convolution layers

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

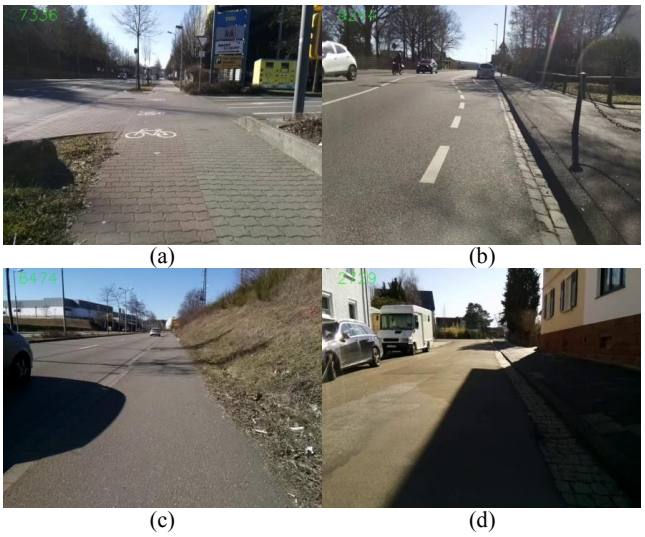


Fig. 2. Dataset examples for lane following with different environments, (a) bike path (indicated by red color) shared with pedestrians (b) marked in-roadway bike lane (c) separated bike lane (d) no bicycle infrastructure

was taken during a very early stage of the work in which the mentioned residual building blocks led to improved accuracy and shorter training times for the networks.

Leaky ReLU is used as activation function in every layer except the output layers, which use hyperbolic tangent functions to be able to represent positive and negative changes. Furthermore batch normalization is performed after each convolution layer. After each fully connected layer, dropout is used with 30% dropout rate.

The fully connected models only have the sensor path. Therefore the Concat block is dropped. Due to not having the visual path, these models are intended to show the improvements introduced by using visual data for the trajectory prediction.

E. Training Details

The used per-sample loss function for training is the Mean Squared Error (MSE) loss for each of the four maneuvers m (turn left, turn right, lane follow, straight), i.e.

$$\text{MSE} = \sum_{k=0}^{n} (\Delta'\varphi_{m,k} - \Delta\varphi_{m,k})^2 + (\Delta' v_{m,k} - \Delta v_{m,k})^2 \quad (3)$$

with $\Delta'\varphi, \Delta' v$ being the output of the network and $\Delta\varphi, \Delta v$ referring to the groundtruth. The time steps n are equal to 25 in our experiments, resulting from the sample time of $0.2\,\text{s}$ and a prediction horizon of $5\,\text{s}$. For the training the Adam Optimizer is used [19] with an initial learning rate of 0.0001. The used framework for training and testing is TensorFlow.

A very challenging problem in imitation learning is to find a well-balanced distribution of samples for training to ensure that the network is capable of generalizing well and not focusing on very often seen features. This can easily be seen considering the nature of vehicle trajectories as

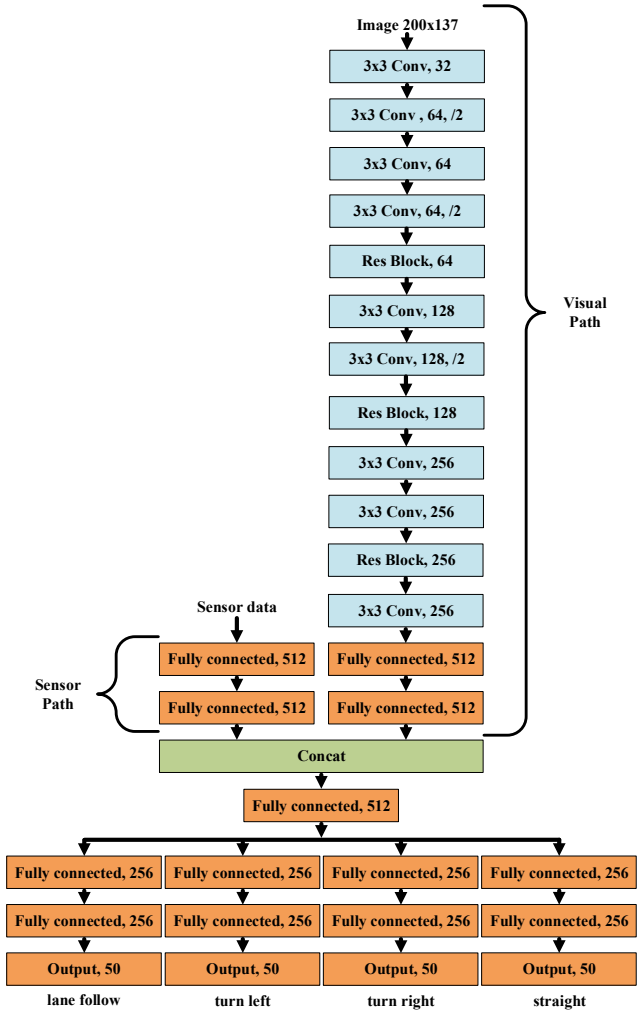


Fig. 3. Network architecture

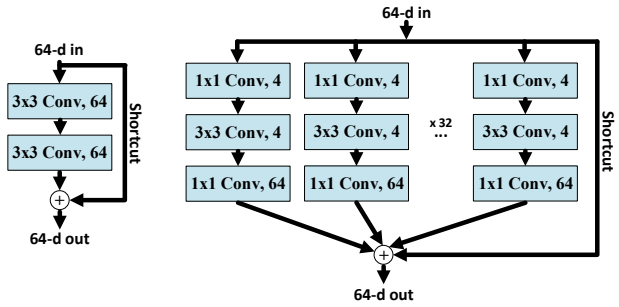


Fig. 4. Left: Building block taken from [17] used in the visual small model versions, Right: Building block taken from [18] used in the visual deep model versions

the most common maneuver is the lane following, which makes up 70% of our overall dataset. Using the branched structure is beneficial since the sub-networks can focus on learning their respective task which circumvents the problem of learning the best fit to often seen maneuvers such as lane following. However, due to the shared visual and sensor path

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

this imbalance in feature learning may still happen. Due to this the same number of randomly selected samples for each maneuver is used in each batch and epoch.

In addition data augmentation is used for the images during training which helps coping with our small dataset. Random transformations such as Gaussian blur, adding salt and pepper noise, altering contrast and brightness are applied to randomly chosen images.

III. Results

For the evaluation, all models were trained several times on 4 hours of driving data and the best result was used for more thorough investigation and comparison. The test set consists of a trip of around 30 minutes in a different area of Kaiserslautern which was not part of the training data.

To compare the results of every model the absolute position error between predicted trajectory and groundtruth trajectory is calculated for every grid point, using the Euclidean distance and taking the mean value over all grid points and all trajectories of the corresponding maneuver.

The results for every model are shown for their respective maneuver in the Figs. 5 to 8.

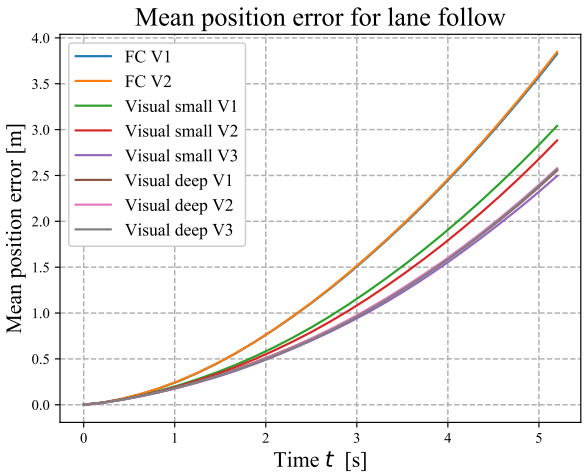


Fig. 5. Mean average position error for lane follow

To better structure the discussion of the results it is organized according to the following topics

- Influence of different inputs and architectures
- General properties of neural networks for long-term trajectory prediction
- Real-time capability

A. Influence of Different Inputs and Architectures

From the results it can easily be seen that for short-term prediction (1 s) all networks behave very similar in terms of performance and then start to diverge.

In general the benefit of using an image in trajectory prediction over pure sensor data lies in its capability to react to the environment and understand the meaning of lanes/lane boundaries when predicting the future trajectories. This can be seen from the results as the networks that use images as

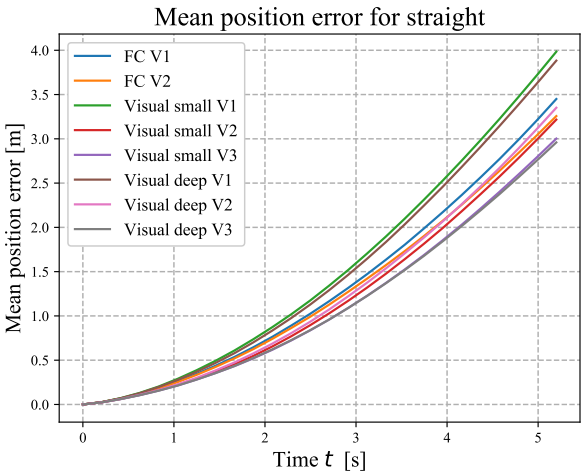


Fig. 6. Mean average position error for driving straight

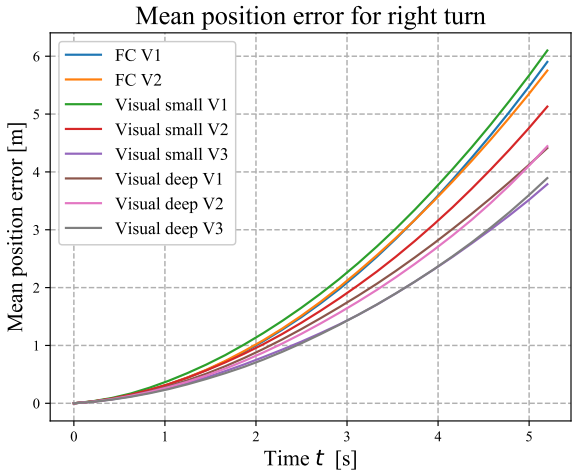


Fig. 7. Mean average position error for right turn

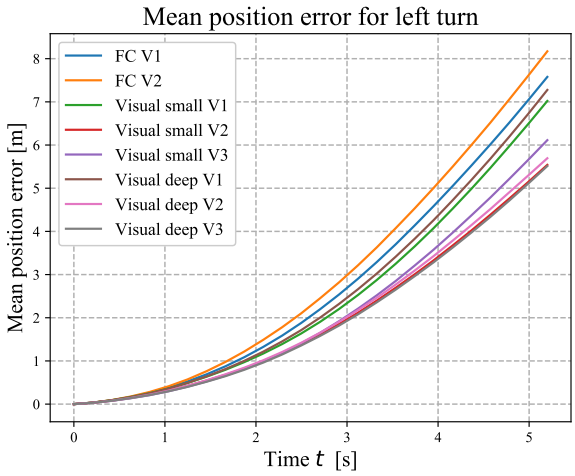


Fig. 8. Mean average position error for left turn

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

input perform better than the fully connected versions. The only exceptions are the visual V1 models which use only the current speed as additional input from sensors. These models perform worse compared to the other models that use images which indicates that the past information is very important. This can be easily seen from the driving "straight" maneuver in Fig. 6. Here the longitudinal dynamics are dominant compared to the lateral as crossing is mostly driving straight. Thus knowing the past accelerations is more beneficial for the prediction than understanding the road layout as there are only minor movements in lateral direction. In contrast to this, the fully connected versions are very close regarding performance and the difference results most likely from the stochastic nature of training a neural network. In general the V3 versions, using all available inputs, perform best, with the small version only performing worse on the left turn compared to the deep version. However, so far it is not clear if the deeper versions have benefits over the smaller versions as, depending on the maneuver, sometimes the deeper and sometimes the smaller version is more accurate. Also the dataset is rather small and thus the deeper versions may not profit from the potentially higher learning capability.

B. General Properties of the Neural Networks for Long-Term Trajectory Prediction

When interpreting the results of long-term predictions a few considerations need to be taken beforehand to understand the challenges and limitations that arise when predicting positions over a long time, especially in the considered urban environment. One difficult situation are traffic lights due to the fact that at time instant $t = 0\,\mathrm{s}$ it cannot be known beforehand that e.g. at $t = 2\,\mathrm{s}$ a traffic light turns red and therefore a stop is necessary. This is resulting in a prediction that moves straight without the stop. However, the groundtruth "knows" the future. Therefore this often results in high errors at the end of the prediction horizon in these situations. Moreover, for methods that only use sensor data, the intention to stop can only anticipated when the driver actually starts to slow down. Yet the models with the image input were able to successfully understand the red traffic light and predicted the upcoming stop correctly at several occasions. Unfortunately as soon as the traffic light was no longer visible due to the camera orientation the stop was no longer predicted.

Similar problems arise considering turning maneuvers as it is not possible to know the situation after a turn, e.g. parked cars or other non-infrastructure related obstacles, beforehand. In this case the models with images have an advantage over the ones without as they adapt as soon as the obstacles are visible. Another issue with turns is that regardless of the model the exact starting point of the turn is very difficult to determine for a bicycle. Depending on the cyclist the turn can be initiated earlier or later as a bicycle can move more freely in a lane compared to a car due to the significantly smaller width. For the vision-based networks it is also difficult if the road to be entered is not or barely visible due to the camera orientation. This scenario and the resulting trajectory prediction can be seen in Fig. 9 and Figs. 10 to 12, respectively (Note trajectory plots always start at (0,0) as initial position). The following results are taken from the Visual small V3 model due to having the best tradeoff between performance and execution time.

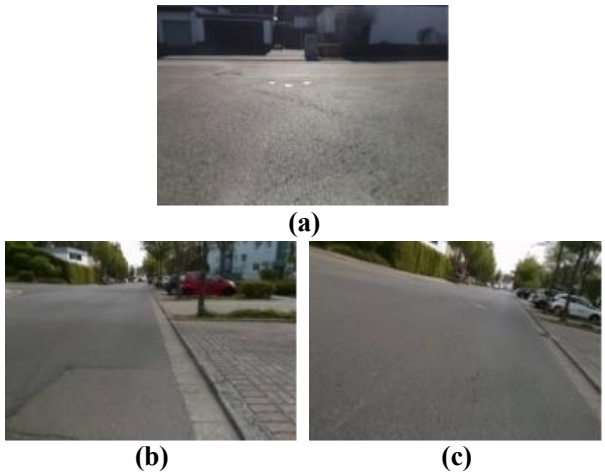


Fig. 9. Input images for three left turn scenarios.

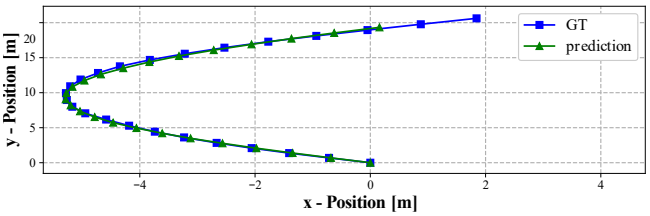


Fig. 10. Result plots for scenario (a) from Fig. 9

Scenario (a) in Fig. 9 shows a left turn into the major road and Fig. 10 shows the resulting trajectory. In this scenario the road boundary (road curb) in front of the house is well visible, providing enough information to anticipate the turning maneuver. From the position plot it can be seen that for this case the qualitative trend of the maneuver is almost perfectly predicted. However, throughout the maneuver the accelerations are over or underestimated by the network as can be seen from the divergence of the markers.

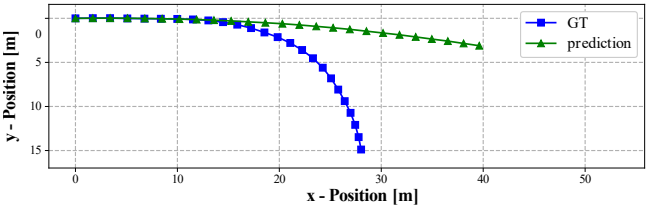


Fig. 11. Result plots for scenario (b) from Fig. 9

Scenario (b) and (c) from Fig. 9 highlight the issues arising when turning left from the major road into the side road. In

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

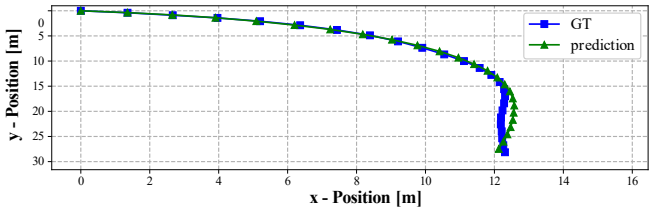


Fig. 12. Result plots for scenario (c) from Fig. 9

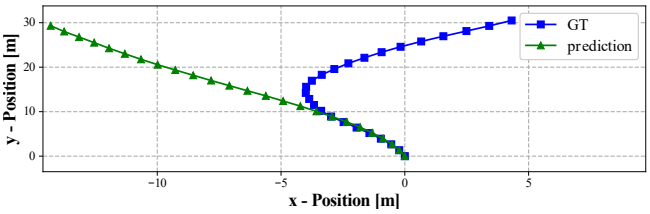


Fig. 14. Result plots for scenario (d) from Fig. 13

(b) the side road is barely visible on the left side and the distance to the turning point is still far. Thus the network does not anticipate the turning maneuver resulting in an very high error, which can be seen from the respective result plot in Fig. 11. In fact the side road entirely vanishes from the image in the following time instance until the turning is initiated. This moment can be seen from Fig. 9 in (c) from the rotated image. The network can already anticipate the initiated turning maneuver and the side arm road boundary is also partially visible. The network is now able to predict the trajectory with good accuracy as can be seen from the results in Fig. 12. To address this issue, wide-angle cameras or additional cameras could be used observing the right and left side of the bicycle as it is common practice in automated vehicles. Fig. 13 shows three scenarios of the lane follow maneuver. Scenario (d) shows a roundabout, (e) shows a right turn with small curvature and a bicycle infrastructure and (f) shows a straight segment with a right turn at the end.

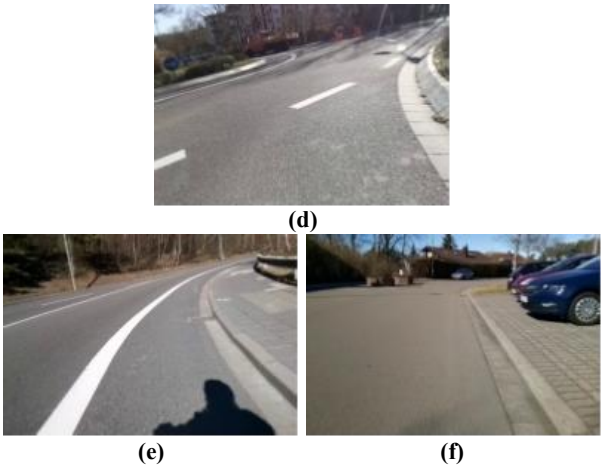


Fig. 13. Lane follow Scenarios with three different infrastructures

The results for the roundabout in scenario (d) are given in Fig. 14. It can be seen that the network predicts the left turn accurately but then expects a straight segment. It is easy to conceive that the right turn to leave the roundabout is impossible to predict without prior knowledge of the road topology and the route.

Fig. 15 shows the results for scenario (e). Here it can be seen that the error is steadily increasing over the prediction horizon. The network has problems to accurately anticipate the speed and heading correctly in the later parts as the view on the infrastructure gets obstructed from the road curb further ahead.

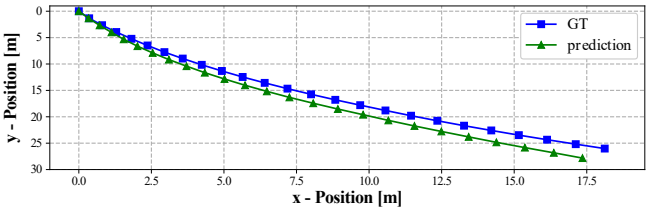


Fig. 15. Result plots for scenario (e) from Fig. 13

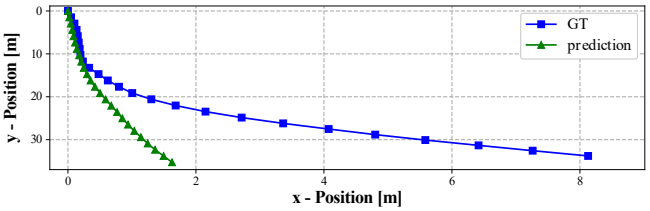


Fig. 16. Result plots for scenario (f) from Fig. 13

The scenario (f) highlights the problem that could also be seen for scenario (e) and in fact is also the dominant problem for right turn maneuvers, namely obstruction of the road layout. From Fig. 13 (f) it can be seen that the road is straight and then turns sharp right which unavoidably leads to an obstruction of the road either by obstacles or the road curb itself (in case of a sidewalk). Due to this the network cannot predict the right turn correctly from the image as can be seen from the results in Fig. 16. The network predicts the straight segment accurately but underestimates the curvature of the right turn. Thus for accurate long-term predictions on roads that include many turns and intersection, using road layout information would be very beneficial.

Another numerics-related problem results from the inability to exactly reach a velocity of zero when predicting stops due to using Δv, resulting in a small velocity, usually around $0.2\,\mathrm{m/s}$ instead of exactly $0\,\mathrm{m/s}$. This leads to an increasing error over the prediction horizon in these scenarios. However, these errors could be handled easily in a validation step after obtaining the network output by interpreting very small velocities as zero.

Despite the problems with long-term predictions at turns, the results are already very promising, considering the

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.

rather small amount of training data and the simplicity of the method and needed data, motivating further research. However, the dynamics and agility as well as the versatile environments for bicycles make this task very difficult and may require significant data collection efforts to ensure a high performance of the trained models.

C. Real-time Capability

As this approach is intended to be used on a bicycle, the execution time of a forward pass of the network is an important metric. To investigate this, the testing program is executed for every model and the results are compared against each other. In Table II the mean execution times as well as the maximum execution times over a full test dataset pass (2797 samples) are given for a desktop PC and an NVIDIA Jetson TX2 platform. The used Desktop PC running Ubuntu 16.04 as operating system, has a 3.6 GHz hexa-core CPU, 32GB Ram and the used GPU is an NVIDIA GeForce GTX 1080Ti.

TABLE II
RESULTS FOR EXECUTION TIME

	execution times in [ms]			
	Desktop		Jetson TX2	
	mean	max	mean	max
FC V1	1.89	4.54	4.78	18.91
FC V2	2.05	6.96	4.83	9.95
Visual small V1	2.95	6.8	24.26	29.19
Visual small V2	6.92	16.12	24.03	29.09
Visual small V3	6.88	10.77	23.88	28.56
Visual deep V1	13.48	36.77	75.6	85.32
Visual deep V2	13.11	20.14	75.16	80.53
Visual deep V3	13.44	18.98	75.17	87.41

From the results it can be seen that the execution time is in general below 100 ms on the Jetson TX2 platform. Considering the given 0.2 s sample time of the system, this is fast enough to be used in ADAS and also leaves some buffer for possibly higher worst-case execution times.

IV. CONCLUSION

In this work we analyzed the usage of several neural networks with different inputs for long-term prediction in an urban environment. The presented networks were trained on real driving data gathered from a bicycle. The results show that despite the very versatile and difficult environment present in everyday driving with the bicycle, the networks already are capable of predicting different maneuvers with good accuracy in short to midterm range ($1-3$ s). For long-term prediction the lack of the future road layout information proves difficult in some scenarios with turns. In addition the results showed that using images as input along with the past movement information significantly outperformed fully connected neural networks for long-term prediction. The real-time capability strongly depends on the application. Yet the small networks have execution times below 30 ms on a Jetson TX2 platform and thus can operate within the often desired 30 fps range of cameras. Future work includes the usage of recurrent neural networks which are expected to further improve accuracy, especially for turns.

REFERENCES

[1] Y. Hu, W. Zhan, and M. Tomizuka, "A framework for probabilistic generic traffic scene prediction," in *Proceedings of the 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2790–2796.

[2] H. Yeping, Z. Wei, and T. Masayoshi, "Probabilistic prediction of vehicle semantic intention and motion," in *Proceedings of the 2018 IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 307–313.

[3] R. Toledo-Moreo and M. A. Zamora-Izquierdo, "IMM-based lane-change prediction in highways with low-cost GPS/INS," *IEEE Transactions on Intelligent Transportation Systems*, vol. 10, no. 1, pp. 180–185, 2009.

[4] J. Li, W. Zhan, Y. Hu, and M. Tomizuka, "Generic tracking and probabilistic prediction framework and its application in autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3634–3649, 2019.

[5] "Verkehr, Verkehrsunfälle," Statistisches Bundesamt, Wiesbaden, Tech. Rep. Fachserie 8 Reihe 7, 2018.

[6] K. Tang, S. Zhu, Y. Xu, and F. Wang, "Modeling drivers' dynamic decision-making behavior during the phase transition period: An analytical approach based on hidden Markov model theory," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 1, pp. 206–214, 2015.

[7] Q. Tran and J. Firl, "Online maneuver recognition and multimodal trajectory prediction for intersection assistance using non-parametric regression," in *Proceedings of IEEE Intelligent Vehicles Symposium (IV)*, 2014, pp. 918–923.

[8] M. Goldhammer, S. Köhler, S. Zernetsch, K. Doll, B. Sick, and K. Dietmayer, "Intentions of vulnerable road users–detection and forecasting by means of machine learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. PP, pp. 1–11, 2019.

[9] G. S. Aoude, V. R. Desaraju, L. H. Stephens, and J. P. How, "Behavior classification algorithms at intersections and validation using naturalistic data," in *Proceedings of the 2011 IEEE Intelligent Vehicles Symposium (IV)*, 2011, pp. 601–606.

[10] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *ROBOMECH Journal*, vol. 1, no. 1, pp. 1–14, 2014.

[11] D. Ridel, E. Rehder, M. Lauer, C. Stiller, and D. Wolf, "A literature review on the prediction of pedestrian behavior in urban scenarios," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 3105–3112.

[12] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras, "Human motion trajectory prediction: A survey," *The International Journal of Robotics Research*, vol. 39, no. 8, pp. 895–935, 2020.

[13] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to end learning for self-driving cars," arXiv:1604.07316, 2016.

[14] F. Codevilla, M. Miiller, A. Lopez, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[15] M. Müller, A. Dosovitskiy, B. Ghanem, and V. Koltun, "Driving policy transfer via modularity and abstraction," *arXiv:1804.09364*, 2018.

[16] S. Chowdhuri, T. Pankaj, and K. Zipser, "MultiNet: Multi-modal multi-task learning for autonomous driving," in *Proceedings of the 2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2019.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[18] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1492–1500.

[19] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," ICLR, 2015.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:31:57 UTC from IEEE Xplore. Restrictions apply.